Exercise Sheet 2: Multi-tape Turing Machines

Formal definition

Definition 1 (Multi-tape Turing machine)**.** Fix an integer $k \geq 1$. A k-tape Turing machine is a tuple

$$M \;=\; (Q, \Sigma, k, \delta)$$

with the following components:

- Q is a finite, nonempty set of states, and contains two special states, the start state q_0 and the halting state q_{halt}.
- Σ is a finite alphabet. Each tape cell either contains a symbol from Σ or is empty (blank), written $\square$; we set the tape alphabet $\Gamma \coloneqq \Sigma \cup \{\square\}$ and require $\square \notin \Sigma$.
- There are k two-way infinite tapes, each equipped with a single read/write head.
- The transition function is

$$\delta : (Q \setminus \{q_{\text{halt}}\}) \times \Gamma^k \;\longrightarrow\; Q \times \Gamma^k \times \{L, R, S\}^k.$$

A configuration of M is a tuple $C = (q, T_1, \ldots, T_k, i_1, \ldots, i_k) \in Q \times (\Gamma^*)^k \times \mathbb{N}^k$, consisting of the current state together with, for each tape $i \in \{1, \ldots, k\}$, the tape contents (a finite word over Γ) and the head position (an index). If

$$\delta(q, a_1, \ldots, a_k) \;=\; (q', b_1, \ldots, b_k, d_1, \ldots, d_k),$$

then this means that, whenever M is in state q, and sees symbols $a_1, \ldots, a_k$ under the tape heads, then, in one step, M:

- writes b_i on tape i (overwriting a_i) for each $i = 1, \ldots, k$,

- moves head i one cell left if $d_i = L$, one cell right if $d_i = R$, or does not move if $d_i = S$,
- and changes state to q'.

The machine halts when it enters state q_{halt} (i.e., δ is not applied in q_{halt}).

Each k-tape Turing machine M thus induces a map $(\Sigma^*)^k \to (\Sigma^*)^k \cup \{\perp\}$, as follows:

- The input $(x_1, \ldots, x_k) \in (\Sigma^*)^k$ is written in each tape, and the tape head is placed on the leftmost symbol of x_i;
- The machine is run one step at a time (as just described), until it halts; if it doesn't halt, the output is $M(x_1, \ldots, x_k) = \perp$.
- If it halts, the output is the contents $y_1, \ldots, y_k$ of the tapes after halting.
- The *computation of M on input $x \in \Sigma^*$* is the (possibly infinite) sequence of configurations $C_0, C_1, \ldots$ obtained by the above procedure.

When we say that a function $f : (\{0,1\}^*)^d \to \{0,1\}^*$ is computable, we mean that there exists a Turing machine M, over some alphabet Σ containing $\{0,1\}$, and with $k > d$ tapes, such that $f(x_1, \ldots, x_k) = (M(x_1, \ldots, x_k, \lambda, \ldots, \lambda))_k$ (i.e. the inputs are given in the first d tapes, and the output is written in the last tape. We then say that a function $f : \mathbb{N}^d \to \mathbb{N}$ is computable if this holds for the function $f : (\{0,1\}^*)^d \to \{0,1\}^*$ which encodes all the numbers in binary.

1. Show that the following functions are computable. Be rigorous, i.e., construct the Turing machine completely. But also, include explanations of what your states are meant to be doing.

(**a**) (sum) $f : \mathbb{N}^2 \to \mathbb{N}$ given by $f(a, b) = a + b$.

(**b**) (select) $f : \{0, 1\} \times \mathbb{N}^2 \to \mathbb{N}$ given by $f(z, a, b) = (1 - z)a + zb$ (i.e., outputs a if $z = 0$ and b if $z = 1$).

(**c**) (max) $f : \mathbb{N}^2 \to \mathbb{N}$ given by $f(a, b) = \max(a, b)$.

(**d**) (less-than) $f : \mathbb{N}^2 \to \{0, 1\}$ given by $f(a, b) = 1$ if $a < b$ and 0 otherwise.

(**e**) (truncated subtraction) $f : \mathbb{N}^2 \to \mathbb{N}$ given by $f(a, b) = \max(a - b, 0)$.

(**f**) (divide-by-2) $f : \mathbb{N} \to \mathbb{N}$ given by $f(x) = \lfloor x/2 \rfloor$.

(**g**) (parity) $f : \mathbb{N} \to \{0, 1\}$ given by $f(x) = 1$ if x is odd, 0 otherwise.

(**h**) (palindrome) $f : \{0, 1\}^* \to \{0, 1\}$ given by $f(w) = 1$ if w is a palindrome, 0 otherwise.

(**i**) (multiplication, this takes some work) $f : \mathbb{N}^2 \to \mathbb{N}$ so that $f(ab) = ab$. Everything is in binary — the school multiplication algorithm is simpler because multiplying by $b_i \times 2^{i-1}$ means you add $a0 \ldots 0$ to the current result (with $i - 1$ extra 0s) or not, depending whether $b_i = 1$ or 0. So the algorithm to implement should be:

```
function multiply(a,b):
   c = 0 # c = contents of output tape
   for each digit of b, starting from right to left:
      if b_i = 1:
          c = c + a
      a = a * 2 # this just appends a zero to a
   return c
```

Obviously you should re-use the machine for sum you wrote above.

2. Show that it is possible to combine Turing machines in the following ways. Try to be both rigorous and succinct, use drawing of the various state diagrams.

(**a**) (composition) Show that given two Turing machines M_1 and M_2, one can construct a Turing machine M which first executes M_1, and then M_2. Show that if $f : \mathbb{N}^n \to \mathbb{N}^m$ and $g : \mathbb{N}^m \to \mathbb{N}^k$ are computable, so is $g \circ f$.

(**b**) (if-then-else) Show that given three Turing machines M, M_0 and M_1, we can implement an if-then-else, "if M outputs 0, run M_0, otherwise run M_1". Show that if $f : \mathbb{N}^n \to \{0,1\}$, $g_0, g_1 : \{0,1\}^n \to \{0,1\}^m$ are computable, then so is the function $h(\bar{x}) = g_{f(\bar{x})}(\bar{x})$.

(**c**) (for loop) Show that we can implement a "for" loop. In terms of functions over $\mathbb{N}$, show that if $f : \mathbb{N}^n \to \mathbb{N}$ and $g : \mathbb{N}^{n+k} \to \mathbb{N}^{n+k}$ are computable, then so is the function

$$h(\bar{x}) = \underbrace{g(g(\cdots g(}_{f(\bar{x}) \text{ times}} \bar{x}, 0, \ldots, 0) \cdots)).$$

(**d**) (while loop) Show that if $f : \mathbb{N}^{n+k} \to \{0,1\}$, $g : \mathbb{N}^{n+k} \to \mathbb{N}^k$, and $h : \mathbb{N}^{n+k} \to \mathbb{N}^\ell$ are computable, then so is the function $F : \mathbb{N}^n \to \mathbb{N}^\ell$ defined by the following pseudocode:

```
function F(x_1, ..., x_n):
    y_1 = y_2 = ... = y_k = 0
    while f(x_1, ..., x_n, y_1, ..., y_k) = 1:
        (y_1, ..., y_k) = g(x_1, ..., x_n, y_1, ..., y_k)
    return h(x_1, ..., x_n, y_1, ..., y_k)
```

(**e**) (parallel run) Show that given two Turing machines M_1 and M_2, one can construct a Turing machine M that simulates both M_1 and M_2 in parallel (interleaving their steps), and and outputs the output of whichever machine halted first.

3. The purpose of this exercise is to give you the understanding that essentially any finitary object can be encoded in binary. As far as I am aware, the first person to realize this in a meaningful way was Kurt Gödel, and this fact was instrumental in the proof of his incompleteness theorems.

Your arguments for computability can and should be reasonably high-level ("the machine does this, then it does that"). You better be sure that the machine can indeed behave in the way you claim (e.g. make sure that your algorithm is such that the number of states Q is finite!).

(**a**) Let $\mathcal{S} = (\{0,1\}^*)^*$ be the set of finite sequences of binary strings. Show that there exists a reasonable encoding $f : \mathcal{S} \to \{0,1\}^*$ [1], so that a map $g : \{0,1\}^* \times \mathbb{N} \to \{0,1\}^*$, such that $g(f(x_1, \ldots, x_n), i) = x_i$, is computable. (Don't forget empty sequences and empty strings!)

[1] Encodings do not need to be surjective.

(**b**) Let $\mathcal{G}$ be the set of undirected graphs. Show that there is a reasonable encoding $f : \mathcal{G} \to \{0,1\}^*$, so that the maps $n : \{0,1\}^* \to \mathbb{N}$ and $e : \{0,1\}^* \times \mathbb{N}^2 \to \{0,1\}$ are computable, so that $n(f(G))$ outputs the number of vertices of G, and $e(f(G), i, j)$ outputs 1 if there is an edge between node i and j in G, and outputs 0 otherwise.

(**c**) Let $\mathcal{T}$ be the set of finite, rooted, ordered trees (trees with a root, and the children of each non-leaf node are ordered, so there is the "first" child up to the "last" child). Devise some encoding $f : \mathcal{T} \to \{0,1\}^*$ so that the map $N : \{0,1\}^* \times \mathbb{N}^* \to \{0,1\}$ is computable, with the property that $N(f(T), i_1, \ldots, i_k) = 1$ if the root has an i_1-th child, who has an i_2-th child, *etc*, i.e. if $(i_1, \ldots, i_k)$ is a valid path on the tree.

(**d**) Let $\mathcal{F}$ be the set of all *finite* functions $\varphi : \{1, 2, \ldots, n\} \to \{1, 2, \ldots, m\}$ for some $n, m \in \mathbb{N}$. Show that there exists an encoding $f : \mathcal{F} \to \{0,1\}^*$ so that the map $g : \{0,1\}^* \times \mathbb{N} \to \mathbb{N}$, with $g(f(\varphi), i) = \varphi(i)$, is computable.

Solution to Exercise 1 (a)

We construct a 3-tape Turing machine that computes addition on binary inputs:

- Tape 1 contains the binary encoding of a (most significant bit first).
- Tape 2 contains the binary encoding of b (most significant bit first).
- Tape 3 is initially blank and will hold the output.

We use $\Sigma = \{0, 1\}$, tape alphabet $\Gamma = \Sigma \cup \{\square\}$.

States:

$$Q = \{q_0,\ q_{\text{seek1}},\ q_{\text{seek2}},\ q_{\text{add0}},\ q_{\text{add1}},\ q_{\text{halt}}\}.$$

- q_0: move input heads to the least significant bits (rightmost symbols).
- q_{seek1}: Tape 1 positioned; finish positioning Tape 2.
- q_{seek2}: Tape 2 positioned; finish positioning Tape 1.
- q_{add0}: addition with carry 0.
- q_{add1}: addition with carry 1.
- q_{halt}: designated halting state.

Transition notation: for $x, y, z \in \Gamma$, we write

$$\delta(q,\ x, y, z)\ =\ (q',\ x', y', z',\ d_1, d_2, d_3),$$

where x' is written on Tape 1 (etc.), and $d_i \in \{L, R, S\}$.

Remarks:

- During seeking, we keep Tape 3 stationary and blank.
- During addition we do not alter input tapes (we rewrite the same symbol we read).
- The output is written least-significant-bit first on Tape 3 (from right to left as heads move left).

Phase 1: Seek rightmost input bits

Move each input head to the cell of its rightmost bit (one cell left of the first $\square$ on that tape).

While both tapes read a bit, move right on both:

$$\delta(q_0,\ 0,0,\square) = (q_0,\ 0,0,\square,\ R,R,S),$$
$$\delta(q_0,\ 0,1,\square) = (q_0,\ 0,1,\square,\ R,R,S),$$
$$\delta(q_0,\ 1,0,\square) = (q_0,\ 1,0,\square,\ R,R,S),$$
$$\delta(q_0,\ 1,1,\square) = (q_0,\ 1,1,\square,\ R,R,S).$$

If Tape 1 hits $\square$ first, step it left once and finish seeking on Tape 2:

$$\delta(q_0,\ \square,0,\square) = (q_{\text{seek1}},\ \square,0,\square,\ L,S,S),$$
$$\delta(q_0,\ \square,1,\square) = (q_{\text{seek1}},\ \square,1,\square,\ L,S,S),$$
$$\delta(q_{\text{seek1}},\ 0,0,\square) = (q_{\text{seek1}},\ 0,0,\square,\ S,R,S),$$
$$\delta(q_{\text{seek1}},\ 1,0,\square) = (q_{\text{seek1}},\ 1,0,\square,\ S,R,S),$$
$$\delta(q_{\text{seek1}},\ 0,1,\square) = (q_{\text{seek1}},\ 0,1,\square,\ S,R,S),$$
$$\delta(q_{\text{seek1}},\ 1,1,\square) = (q_{\text{seek1}},\ 1,1,\square,\ S,R,S),$$
$$\delta(q_{\text{seek1}},\ 0,\square,\square) = (q_{\text{add0}},\ 0,\square,\square,\ S,L,S),$$
$$\delta(q_{\text{seek1}},\ 1,\square,\square) = (q_{\text{add0}},\ 1,\square,\square,\ S,L,S).$$

If Tape 2 hits $\square$ first:

$$\delta(q_0,\ 0,\square,\square) = (q_{\text{seek2}},\ 0,\square,\square,\ S,L,S),$$
$$\delta(q_0,\ 1,\square,\square) = (q_{\text{seek2}},\ 1,\square,\square,\ S,L,S),$$
$$\delta(q_{\text{seek2}},\ 0,0,\square) = (q_{\text{seek2}},\ 0,0,\square,\ R,S,S),$$
$$\delta(q_{\text{seek2}},\ 0,1,\square) = (q_{\text{seek2}},\ 0,1,\square,\ R,S,S),$$
$$\delta(q_{\text{seek2}},\ 1,0,\square) = (q_{\text{seek2}},\ 1,0,\square,\ R,S,S),$$
$$\delta(q_{\text{seek2}},\ 1,1,\square) = (q_{\text{seek2}},\ 1,1,\square,\ R,S,S),$$
$$\delta(q_{\text{seek2}},\ \square,0,\square) = (q_{\text{add0}},\ \square,0,\square,\ L,S,S),$$
$$\delta(q_{\text{seek2}},\ \square,1,\square) = (q_{\text{add0}},\ \square,1,\square,\ L,S,S).$$

Edge case (both inputs empty):

$$\delta(q_0,\square,\square,\square) = (q_{\text{add0}},\square,\square,\square,\ L,L,S)$$

Phase 2: Addition with carry

Let blanks be treated as zeros on input tapes. We now give the full set of rows for all possible combinations of input bits and carry. In each case we rewrite the same symbols on Tapes 1 and 2 (i.e., preserve inputs), write the sum bit on Tape 3, then move left on all three tapes.

Carry 0 state (q_{add0}):

$$\delta(q_{\text{add0}},\ 0,0,\square) = (q_{\text{add0}},\ 0,0,0,\ L,L,L) \quad (0{+}0{+}0 \Rightarrow \text{bit } 0, \text{ carry } 0),$$
$$\delta(q_{\text{add0}},\ 0,1,\square) = (q_{\text{add0}},\ 0,1,1,\ L,L,L) \quad (0{+}1{+}0 \Rightarrow \text{bit } 1, \text{ carry } 0),$$
$$\delta(q_{\text{add0}},\ 1,0,\square) = (q_{\text{add0}},\ 1,0,1,\ L,L,L) \quad (1{+}0{+}0 \Rightarrow \text{bit } 1, \text{ carry } 0),$$
$$\delta(q_{\text{add0}},\ 1,1,\square) = (q_{\text{add1}},\ 1,1,0,\ L,L,L) \quad (1{+}1{+}0 \Rightarrow \text{bit } 0, \text{ carry } 1),$$
$$\delta(q_{\text{add0}},\ \square,0,\square) = (q_{\text{add0}},\ \square,0,0,\ L,L,L) \quad (\square \text{ as } 0),$$
$$\delta(q_{\text{add0}},\ 0,\square,\square) = (q_{\text{add0}},\ 0,\square,0,\ L,L,L) \quad (\square \text{ as } 0),$$
$$\delta(q_{\text{add0}},\ \square,1,\square) = (q_{\text{add0}},\ \square,1,1,\ L,L,L) \quad (\square \text{ as } 0),$$
$$\delta(q_{\text{add0}},\ 1,\square,\square) = (q_{\text{add0}},\ 1,\square,1,\ L,L,L) \quad (\square \text{ as } 0),$$
$$\delta(q_{\text{add0}},\ \square,\square,\square) = (q_{\text{halt}},\ \square,\square,\square,\ S,S,S) \quad (\text{no bits and no carry: halt}).$$

Carry 1 state (q_{add1}):

$$\delta(q_{\text{add1}},\ 0,0,\square) = (q_{\text{add0}},\ 0,0,1,\ L,L,L) \quad (0{+}0{+}1 \Rightarrow \text{bit } 1, \text{ carry } 0),$$
$$\delta(q_{\text{add1}},\ 0,1,\square) = (q_{\text{add1}},\ 0,1,0,\ L,L,L) \quad (0{+}1{+}1 \Rightarrow \text{bit } 0, \text{ carry } 1),$$
$$\delta(q_{\text{add1}},\ 1,0,\square) = (q_{\text{add1}},\ 1,0,0,\ L,L,L) \quad (1{+}0{+}1 \Rightarrow \text{bit } 0, \text{ carry } 1),$$
$$\delta(q_{\text{add1}},\ 1,1,\square) = (q_{\text{add1}},\ 1,1,1,\ L,L,L) \quad (1{+}1{+}1 \Rightarrow \text{bit } 1, \text{ carry } 1),$$
$$\delta(q_{\text{add1}},\ \square,0,\square) = (q_{\text{add0}},\ \square,0,1,\ L,L,L) \quad (\square \text{ as } 0, \text{ carry does not persist}),$$
$$\delta(q_{\text{add1}},\ 0,\square,\square) = (q_{\text{add0}},\ 0,\square,1,\ L,L,L) \quad (\square \text{ as } 0, \text{ carry does not persist}),$$
$$\delta(q_{\text{add1}},\ \square,1,\square) = (q_{\text{add1}},\ \square,1,0,\ L,L,L) \quad (\square \text{ as } 0, \text{ carry persists}),$$
$$\delta(q_{\text{add1}},\ 1,\square,\square) = (q_{\text{add1}},\ 1,\square,0,\ L,L,L) \quad (\square \text{ as } 0, \text{ carry persists}),$$
$$\delta(q_{\text{add1}},\ \square,\square,\square) = (q_{\text{halt}},\ \square,\square,1,\ S,S,S) \quad (\text{final carry 1: write it and halt}).$$

Solution to Exercise 1 (b)

We construct a 4-tape Turing machine that computes the selector $f(z,a,b) = (1-z)a + zb$ for $z \in \{0,1\}$ and $a,b \in \mathbb{N}$.

- Tape 1 contains the bit z (either 0 or 1), head at its leftmost cell.
- Tape 2 contains the binary encoding of a (MSB first), head at the leftmost bit.

- Tape 3 contains the binary encoding of b (MSB first), head at the leftmost bit.
- Tape 4 is initially blank and will hold the output.

Use $\Sigma = \{0, 1\}$ and $\Gamma = \Sigma \cup \{\square\}$.

States:

$$Q \;=\; \{q_0,\, q_{\text{copya}},\, q_{\text{copyb}},\, q_{\text{halt}}\}.$$

- q_0: Read z and branch to the corresponding copy routine.
- q_{copya}: Copy Tape 2 to Tape 4 (when $z = 0$).
- q_{copyb}: Copy Tape 3 to Tape 4 (when $z = 1$).
- q_{halt}: Halting state.

Transitions: We write

$$\delta(q,\; x_1, x_2, x_3, x_4) \;=\; (q',\; x_1', x_2', x_3', x_4',\; d_1, d_2, d_3, d_4).$$

Branch on z:

$$\delta(q_0,\; 0, x, y, \square) = (q_{\text{copya}},\; 0, x, y, \square,\; S, S, S, S) \quad \text{for } x, y \in \{0, 1\},$$

$$\delta(q_0,\; 1, x, y, \square) = (q_{\text{copyb}},\; 1, x, y, \square,\; S, S, S, S) \quad \text{for } x, y \in \{0, 1\}.$$

Copy a (Tape 2) to Tape 4:

$$\delta(q_{\text{copya}},\; z, 0, y, \square) = (q_{\text{copya}},\; z, 0, y, 0,\; S, R, S, R),$$

$$\delta(q_{\text{copya}},\; z, 1, y, \square) = (q_{\text{copya}},\; z, 1, y, 1,\; S, R, S, R),$$

$$\delta(q_{\text{copya}},\; z, \square, y, \square) = (q_{\text{halt}},\; z, \square, y, \square,\; S, S, S, S).$$

Copy b (Tape 3) to Tape 4:

$$\delta(q_{\text{copyb}},\; z, x, 0, \square) = (q_{\text{copyb}},\; z, x, 0, 0,\; S, S, R, R),$$

$$\delta(q_{\text{copyb}},\; z, x, 1, \square) = (q_{\text{copyb}},\; z, x, 1, 1,\; S, S, R, R),$$

$$\delta(q_{\text{copyb}},\; z, x, \square, \square) = (q_{\text{halt}},\; z, x, \square, \square,\; S, S, S, S).$$

Explanation:

- In q_0 the machine reads z and commits to copying either a or b.
- The selected input is then copied symbol-by-symbol to Tape 4 while advancing on that input and the output; when a blank is seen on the selected input, the machine halts.
- Heads are assumed to start on the leftmost position of each input, so no rewinding is necessary.

Solution to Exercise 1 (c)

We construct a 3-tape Turing machine that computes the maximum of two natural numbers given in binary:

- Tape 1 contains the binary encoding of a (most significant bit first, leftmost cell, and the first bit is 1).
- Tape 2 contains the binary encoding of b (same format).
- Tape 3 is initially blank and will hold the output (the maximum).

High-level idea:

1. First, compare the lengths of the two inputs by moving both heads to the right until one hits a blank. If one input is longer, copy it to the output and halt.
2. If both inputs are the same length, rewind both heads to the leftmost bit, then scan both tapes simultaneously, comparing bits. At the first position where the bits differ, copy the input with a 1 at that position to the output and halt.
3. If all bits are equal, copy either input to the output (they are equal).

States:

$$Q = \{q_0, q_{\text{seek}}, q_{\text{rewind}}, q_{\text{cmp}}, q_{\text{copy1}}, q_{\text{copy2}}, q_{\text{halt}}\}$$

- q_0: Initial state, move both heads right to compare lengths.
- q_{seek}: One head hit blank, determine which input is longer.
- q_{rewind}: Both inputs same length, rewind both heads to the left.
- q_{cmp}: Compare bits one by one.
- q_{copy1}: Copy input 1 to output.
- q_{copy2}: Copy input 2 to output.
- q_{halt}: Halting state.

Transitions:

- In q_0, move both heads right as long as both see a bit:

$$\delta(q_0, x, y, \square) = (q_0, x, y, \square, R, R, S) \quad \text{for } x, y \in \{0, 1\}$$

- If Tape 1 hits blank first:

$$\delta(q_0, \square, y, \square) = (q_{\text{seek}}, \square, y, \square, S, S, S)$$

  In q_{seek}, if Tape 2 is not blank, input 2 is longer:

$$\delta(q_{\text{seek}}, \square, y, \square) = (q_{\text{copy2}}, \square, y, \square, S, S, S) \quad \text{for } y \in \{0, 1\}$$

- If Tape 2 hits blank first:

$$\delta(q_0, x, \square, \square) = (q_{\text{seek}}, x, \square, \square, S, S, S)$$

  In q_{seek}, if Tape 1 is not blank, input 1 is longer:

$$\delta(q_{\text{seek}}, x, \square, \square) = (q_{\text{copy1}}, x, \square, \square, S, S, S) \quad \text{for } x \in \{0, 1\}$$

- If both hit blank at the same time (same length):

$$\delta(q_0, \square, \square, \square) = (q_{\text{rewind}}, \square, \square, \square, L, L, S)$$

  In q_{rewind}, move both heads left until both see blank (rewind to leftmost bit):

$$\delta(q_{\text{rewind}}, x, y, \square) = (q_{\text{rewind}}, x, y, \square, L, L, S) \quad \text{for } x, y \in \{0, 1\}$$

  When both see blank, move right to first bit and start comparison:

$$\delta(q_{\text{rewind}}, \square, \square, \square) = (q_{\text{cmp}}, \square, \square, \square, R, R, S)$$

- In q_{cmp}, compare bits:
  - If $x = y$, move right and copy to output:

$$\delta(q_{\text{cmp}}, x, x, \square) = (q_{\text{cmp}}, x, x, x, R, R, S) \quad \text{for } x \in \{0, 1\}$$

  - If $x = 1, y = 0$, input 1 is greater:

$$\delta(q_{\text{cmp}}, 1, 0, \square) = (q_{\text{copy1}}, 1, 0, \square, S, S, S)$$

- If $x = 0, y = 1$, input 2 is greater:

$$\delta(q_{\text{cmp}}, 0, 1, \square) = (q_{\text{copy2}}, 0, 1, \square, S, S, S)$$

- If both see blank (all bits equal), the entire input has been copied:

$$\delta(q_{\text{cmp}}, \square, \square, \square) = (q_{\text{halt}}, \square, \square, \square, S, S, S)$$

- In q_{copy1}, copy Tape 1 to Tape 3, moving right, until blank:

$$\delta(q_{\text{copy1}}, x, y, \square) = (q_{\text{copy1}}, x, y, x, R, S, R) \quad \text{for } x \in \{0, 1\}$$

When blank is reached:

$$\delta(q_{\text{copy1}}, \square, y, \square) = (q_{\text{halt}}, \square, y, \square, S, S, S)$$

- In q_{copy2}, copy Tape 2 to Tape 3, moving right, until blank:

$$\delta(q_{\text{copy2}}, x, y, \square) = (q_{\text{copy2}}, x, y, y, S, R, R) \quad \text{for } y \in \{0, 1\}$$

When blank is reached:

$$\delta(q_{\text{copy2}}, x, \square, \square) = (q_{\text{halt}}, x, \square, \square, S, S, S)$$

Explanation:

- The first pass determines if one input is longer, in which case it is the maximum.
- If both are the same length, the machine rewinds and compares bit by bit, outputting the input with the first 1 at a differing position.
- If all bits are equal, either input can be output.

Solution to Exercise 2 (a)

We show how to combine two multi-tape Turing machines sequentially.

Let $M_1 = (Q_1, \Sigma_1, k_1, \delta_1)$ compute f and $M_2 = (Q_2, \Sigma_2, k_2, \delta_2)$ compute g. Replace both alphabets by $\Sigma := \Sigma_1 \cup \Sigma_2$ (so both machines work over Σ) and pad each to $k := \max\{k_1, k_2\}$ tapes by letting the "extra" tapes be ignored (heads stay put and symbols unchanged). Assume M_1 takes its n inputs on some fixed set of tapes and, upon halting, leaves its m outputs on a fixed set of "interface" tapes; M_2 expects its inputs on exactly those interface tapes and, upon halting, leaves its k outputs on some fixed output tapes. (If necessary, we first rename tapes so that these conventions match.)

Construction of M (run M_1 then M_2):

- States. Take disjoint copies of the states of M_1 and M_2 (rename to avoid clashes) and add two glue states: $Q = Q_1' \dot\cup Q_2' \dot\cup \{q_{\text{bridge}}, q_{\text{halt}}\}$. Identify the start state of M with the (renamed) start state of M_1. Redirect the (renamed) halting state of M_2 to q_{halt}.

- Simulating M_1. For every transition of M_1, copy it verbatim on the renamed states and k tapes. For each transition of M_1 that would go to its halting state, keep the same tape writes and head moves, but go to q_{bridge} instead.

- Bridge (reposition heads). In q_{bridge}, for each interface tape, run the standard rewind routine: move left until seeing $\square$, then move right once, leaving the head at the leftmost symbol; leave all other tapes and symbols unchanged. After all interface tapes are rewound, jump to the (renamed) start state of M_2.

- Simulating M_2. Copy all transitions of M_2 on the renamed states; when M_2 would enter its halting state, go to q_{halt}.

Correctness. On input $\bar{x}$, the M_1-phase produces $f(\bar{x})$ on the interface tapes. The bridge only repositions heads, so the contents are unchanged but prepared as valid inputs to M_2. The M_2-phase then computes $g(f(\bar{x}))$ and halts, so M computes $g \circ f$. Therefore, if $f : \mathbb{N}^n \to \mathbb{N}^m$ and $g : \mathbb{N}^m \to \mathbb{N}^k$ are computable, so is $g \circ f$ (under the binary encodings fixed in the sheet).

Solution to Exercise 2 (b)

We implement an if-then-else: run a predicate machine M that outputs a single bit; if the bit is 0 run M_0, otherwise run M_1.

Let $M = (Q_M, \Sigma_M, k_M, \delta_M)$ compute $f : \mathbb{N}^n \to \{0,1\}$, and let $M_0 = (Q_0, \Sigma_0, k_0, \delta_0)$ and $M_1 = (Q_1, \Sigma_1, k_1, \delta_1)$ compute $g_0, g_1 : \mathbb{N}^n \to \mathbb{N}^m$. Standardize as in 2(a): take a common alphabet $\Sigma = \Sigma_M \cup \Sigma_0 \cup \Sigma_1$, and a common tape count $k = \max\{k_M, k_0, k_1\}$ (padding unused tapes with stationary heads). Fix a block I of n input tapes and a block O of m output tapes. Let τ_{dec} be a designated decision tape on which M writes its single-bit output. We also reserve a backup block I' of n tapes to hold a protected copy of the input.

Construction of $M^\star$ (if-then-else on f, g_0, g_1):

- States. Use disjointly renamed copies of the states of M, M_0, M_1, and add glue states

  $$\{q_{\text{copy}},\ q_{\text{branch}},\ q_{\text{prep0}},\ q_{\text{prep1}},\ q_{\text{halt}}\}.$$

  The start state of $M^\star$ is q_{copy}. Redirect the halting states of the renamed M_0 and M_1 to q_{halt}.

- Protect inputs. In q_{copy}, copy the contents of each tape in I to the corresponding tape in I' (symbol-by-symbol left to right). When all n copies are complete, rewind heads on I to their leftmost symbols and jump to the (renamed) start state of M.

- Simulate M. Copy all transitions of M. When M would enter its halting state, keep the same writes and moves (so that τ_{dec} holds 0 or 1), but go to q_{branch} instead.

- Branch. In q_{branch}, read the symbol under the head on τ_{dec}:

  - If it is 0, go to q_{prep0}.
  - If it is 1, go to q_{prep1}.

- Prepare inputs for the selected branch. In q_{prep0} (respectively q_{prep1}), rewind all heads on the backup block I' to their leftmost symbols, and clear/rewind the output block O as needed. Then jump to the (renamed) start state of M_0 (respectively M_1), after having relabeled I' as the input block for that machine and O as its output block. Simulate the selected machine until it halts, then enter q_{halt}.

Correctness. On input $\bar{x}$, the copy phase preserves a pristine copy $\bar{x}$ on I'. The M-phase computes $f(\bar{x})$ and leaves its bit on τ_{dec}. The branch reads this bit and starts either the M_0- or M_1-phase, both of which receive $\bar{x}$ from the protected block I', producing $g_{f(\bar{x})}(\bar{x})$ on the output block O. Hence $M^\star$ computes $h(\bar{x}) = g_{f(\bar{x})}(\bar{x})$, proving that if f, g_0, g_1 are computable, so is h.

Solution to Exercise 3 (a)

Encoding. We use a fixed-length, pairwise code over $\{0,1\}^*$:

$$\mathrm{enc}(0) = 00, \qquad \mathrm{enc}(1) = 01, \qquad \mathrm{sep} = 11.$$

For a single binary string $w = b_1 b_2 \cdots b_m \in \{0,1\}^*$, define

$$\mathrm{code}(w) \;=\; \mathrm{enc}(b_1)\,\mathrm{enc}(b_2)\cdots\mathrm{enc}(b_m),$$

so $\mathrm{code}(\lambda) = \lambda$. For a finite sequence $(x_1, \ldots, x_n) \in (\{0,1\}^*)^*$, set

$$f(x_1, \ldots, x_n) \;=\; \mathrm{code}(x_1)\,\mathrm{sep}\,\mathrm{code}(x_2)\,\mathrm{sep}\,\cdots\,\mathrm{sep}\,\mathrm{code}(x_n),$$

and for the empty sequence $()$, define $f() = \lambda$. This is uniquely decodable because payload symbols are exactly the pairs in $\{00, 01\}$, while the delimiter is the distinct pair 11; the unused pair 10 never appears.

Decoding. We construct a 4-tape Turing machine M_g that, on input (s, i) where $s \in \{0,1\}^*$ and $i \in \mathbb{N}$ (binary, MSB first), outputs x_i if $s = f(x_1, \ldots, x_n)$ and $1 \le i \le n$, and outputs λ otherwise. We explicitly use two standard subroutines on Tape 2 (the index): - DEC: decrement a nonzero binary number by 1 (with borrow), leaving 0 unchanged; - EQ1: decide whether a binary number equals 1.

Tapes:

- Tape 1: the encoded sequence s, head at its leftmost symbol;
- Tape 2: the index i in binary (MSB first), head at the leftmost bit;
- Tape 3: output (initially blank);
- Tape 4: work tape (scratch).

Algorithm:

1. If Tape 2 is blank or represents 0, halt with Tape 3 blank (out of range).

2. Position at the start of the first block on Tape 1 (already true initially).

3. While not EQ1(Tape 2):

(**a**) Skip exactly one block on Tape 1: read symbols in pairs.

- If Tape 1 is blank before a delimiter is seen, halt with Tape 3 blank (fewer than i blocks).

- Otherwise, advance by two cells at a time while reading pairs 00 or 01 (payload).
- When the pair 11 (a separator) is seen, advance past it; the head now rests at the start of the next block (which may be empty if another 11 or blank follows).

(**b**) Invoke DEC on Tape 2 (i := i - 1).

4. Copy the next block from Tape 1 to Tape 3:

- Read Tape 1 in pairs. For 00 write 0 to Tape 3; for 01 write 1 to Tape 3 (advance both tapes appropriately after each pair).
- Stop when seeing 11 (the next separator) or blank; in either case halt. If the first pair is already 11 or blank, the i-th string is empty and Tape 3 remains blank.

Correctness. The stream is parsed in disjoint pairs; payload pairs are $\{00, 01\}$ and the unique delimiter is 11, hence block boundaries are unambiguous. Each loop iteration skips exactly one block and decrements i using DEC. When EQ1 holds, the head on Tape 1 is at the start of the i-th block, which is copied bitwise to the output by translating pairs $00 \mapsto 0$ and $01 \mapsto 1$ until the next separator or end of input. If the input ends while skipping with $i > 1$, we correctly report out-of-range by leaving Tape 3 blank. All used routines (pairwise scanning, DEC, EQ1, and bounded copying) are standard multi-tape TM subroutines, so the map $g : \{0,1\}^* \times \mathbb{N} \to \{0,1\}^*$ with $g(f(x_1, \ldots, x_n), i) = x_i$ is computable under this encoding.